Static AST-Based Triage of Socially-Engineered Developer Lures: Proving Obfuscator Rotations Without Execution

Ilja Krasnjanski

Independent researcher

ORCID: 0009-0007-1039-5930

29 October 2026 · Preprint

Abstract

Recruitment lures can conceal malicious JavaScript in clipboard commands, assessment repositories and editor extensions. Their commands are assembled at copy time, strings are encoded in rotated arrays, and process launches resemble those of legitimate developer tools. We present `unk-scanner`, a static triage tool that combines bounded partial evaluation, checksum-based string-array recovery and clipboard-value tracing without executing the input. Rules assign severity from combinations of techniques, and the command-line interface isolates analysis processes and emits SARIF. Evaluation covers 2.8 million files across popular software, recently changed npm packages and known-malicious packages. After one rule adjustment informed by the popular-software corpus, the final scan raised no error-level findings in its 188,901 files outside the original calibration set. A scan of 2.4 million files from 19,976 recently changed npm packages raised two errors, both false positives. Sensitivity was limited: only 62 of 2,122 public malware samples raised warnings, and none raised errors. Retained UNK_DeadDrop artifacts reproduce the expected outputs, including a known missed agent, but they also informed rule design. In a separate bypass test, 18 of 19 single-change variants evaded detection, reduced severity or defeated recovery; one caused incorrect string decoding. On 7,200 samples obfuscated with two javascript-obfuscator releases, every call site the scanner decoded matched the original program. It recovered 87% of hidden strings from the current release and none from the older one, whose decoder shape it does not recognise; a sandboxed deobfuscator recovered 98% of both. A hardened engine, evaluated afterwards on the same corpora, handles all but one of those variants without new error-level findings, but the variants were written by the authors and do not measure resistance to an adaptive adversary. The results establish low error-level alert volume in the evaluated corpora, alongside substantial coverage and evasion limits. The scanner assists inspection of the delivery structures it recognises; it does not provide a general malware verdict.

Keywords: static analysis; JavaScript deobfuscation; software supply chain; ClickFix; editor extensions; developer security

1. Introduction

Supply-chain defences commonly examine packages that execute malicious code at installation or import, including typosquats and compromised dependencies. Recruitment campaigns introduce another route: an assessment page persuades a developer to paste a verification command, or a take-home repository configures the editor to install an extension. The malicious code may run only after this interaction, and its purpose can be difficult to determine from a brief source review.

Three features complicate static inspection. First, the studied ClickFix lures construct commands when the user copies them, combining platform checks, helper functions and command fragments. Reading clipboard-call literals alone misses this construction. Second, obfuscator.io-style extensions replace module names, URLs and configuration strings with decoder calls into encoded, rotated arrays. Third, suspicious process-launch techniques also occur in legitimate tools. Both malware and language clients use `ELECTRON_RUN_AS_NODE` to run helpers through an editor binary, while commercial extensions also use string-array obfuscation.

Dynamic analysis can reveal execution behaviour, but it requires running hostile code and may leave platform-, session- or server-dependent branches unexplored. This paper examines how much of the command construction and hidden configuration can be recovered statically, and how reliably those structures distinguish malicious artifacts from ordinary developer software.

The approach combines a bounded evaluator for a side-effect-free expression subset, string-array recovery using the obfuscator's checksum, and tracing from clipboard writes into local builders and caller arguments. Checksum-based recovery is also used by synchrony [8]; here it shares an evaluator with the detection rules. Severity depends on combinations of techniques rather than obfuscation or Electron use alone. The implementation adds process isolation and SARIF output.

The evaluation measures alert volume, parser coverage, throughput and sensitivity on three registry-derived corpora, then tests recovery and bypasses using retained artifacts and inert variants. A controlled experiment with 7,200 obfuscated samples of known content measures whether recovered strings are correct, and compares recovery with three published deobfuscators. It shows both the practical value of exposing hidden commands and the fragility of structural rules: low error-level alert volume does not imply broad detection coverage.

2. Background and threat model

We consider JavaScript delivered to a developer through social engineering:

- ClickFix pages that substitute a terminal command on copy;
- repositories whose workspace configuration (for example a `runOn: folderOpen` task) runs or installs code;
- editor extensions, sideloaded or published;
- npm packages consumed as part of an assessment.

The UNK_DeadDrop campaign used all of the first three [1, 2]. Related activity is tracked publicly as Contagious Interview (MITRE ATT&CK G1052) [4], first reported as fake job interviews that ask developers to review an npm package [12].

The obfuscator replaces string literals with calls to a decoder, such as `_0x2bac03(0x530,'rkdk',…)`. Wrappers map their arguments to an index and, for the RC4 variant, a key. The decoder subtracts an offset, reads an element of a stored array, decodes it (plain, base64 with a file-specific alphabet, or RC4 over base64), and caches the result. The stored array is rotated. At start-up an immediately invoked function repeatedly decodes a few numeric-looking strings, combines them with `parseInt` into a checksum, and shifts the array until the checksum equals a constant embedded in the file. A wrong rotation does not fail; it silently returns other entries of the same array.

The analyst holds a file, an unpacked package or an extension, and wants to know what it is set up to do without running it. The adversary controls the source and may target the analyser: deep nesting, pathological sizes, code designed to exhaust the parser, or inputs crafted to make an evaluator perform side effects. The corpus evaluation measures existing code; the bypass study separately considers an adversary who knows the rules and changes the input to evade them. Out of scope are code fetched at run time that is not present in the file, native payloads (handled by file-content signatures), and proving reachability.

3. Design

Figure 1 shows the analysis path. Each file is parsed once. An index pass records parent links and function scopes. String-array recovery runs before the rules, so every rule sees recovered values in place of decoder calls, and recovery and rules share one pure evaluator. Parsing, recovery and rules run inside an isolated analysis process (Section 3.6), and only the report leaves it. This section describes the engine evaluated in Sections 5.1 to 5.5 (source hash prefix `caf3f552`). Section 5.6 lists the later changes to evaluation, clipboard sinks, launch escalation, loader linking and rotation recognition; they are not repeated here.

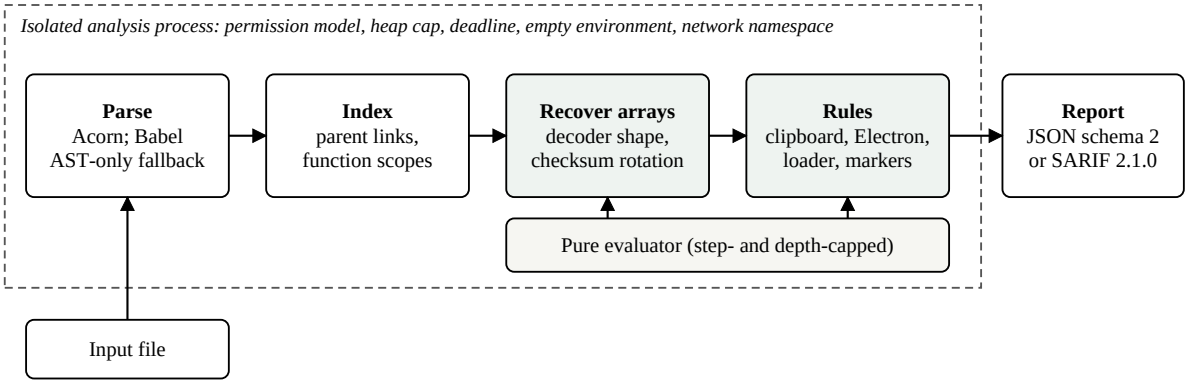


Figure 1. Analysis pipeline. A file is parsed and indexed; obfuscated string arrays are recovered before the rules run; recovery and rules share a step- and depth-capped evaluator that never calls into the input. The dashed boundary is the isolated analysis process; only the report leaves it.

3.1 Parsing

Files are parsed with Acorn as ECMAScript module, then as script. When Acorn rejects a file, `@babel/parser` parses it in AST-only mode with the TypeScript and/or JSX plugins, chosen by file extension or tried in turn, and emits ESTree. The same rules therefore apply to TypeScript sources and JSX components. TypeScript-only wrappers (`as`, `satisfies`, non-null assertions) are transparent to evaluation and member resolution. Location objects are not materialised; positions are computed from offsets only for reported nodes, which lowers peak memory on large bundles. All tree walks are iterative.

3.2 Pure partial evaluation

The evaluator interprets the following:

- literals and templates;
- arithmetic, bitwise, comparison and logical operators on primitives;
- `const` bindings and properties of literal objects;
- string and array indexing and `.length`;
- `parseInt`, `String.fromCharCode` and a fixed list of string methods;
- functions whose body is a single `return` of a supported expression, with parameters bound to argument values.

Any other construct yields *unknown*, which propagates. This includes calls on unknown values, `let`/`var`, assignments, loops, getters, spreads and regular expressions. Each evaluation is capped at 20,000 steps and depth 48, strings at 1 MiB and arrays at 10,000 elements. Scopes are function-level, and a name declared twice in one scope becomes unknown. Unsupported operations remain unknown rather than being delegated to the JavaScript runtime. This restriction prevents the evaluator from executing input-supplied functions; it does not establish that every supported expression is interpreted correctly.

3.3 Proving the rotation of string arrays

An array source is a self-redefining function that holds a string-array literal, or an array variable that rotation code shifts. A decoder is the innermost function that references the source, indexes it and rebases its first parameter (`i = i - OFFSET`). The offset is evaluated. Candidate variants come from the decoder's syntax:

- a 65-character alphabet literal indicates base64;
- a used key parameter indicates RC4;
- otherwise the array is plain.

The scanner then decodes in its own code; the input's decoder is never run.

The rotation loop is found as an immediately invoked function that receives the array or fetches it by name and contains `x.push(x.shift())`. Within its `try` block, the stopping condition is either `if (v === target) break` with `const v = EXPR`, or `if (TARGET == EXPR) break`. Loop parameters are bound to the call's arguments. For each candidate rotation `r = 0 … n−1`, the evaluator computes the checksum expression through the same wrappers and decoders, and takes the first `r` that equals the target. When several decoders share one array, as with a base64 decoder and an RC4 decoder over the same strings, all combinations of their candidate variants are tried under one shared rotation. Algorithm 1 summarises the procedure.

```
Algorithm 1. Rotation by checksum, without executing the input
input:  array source A of n strings; decoders D over A, each with an offset and
        candidate variants; the rotation call F(args), whose try block holds E == target
output: a rotation r, or none
bind F's parameters to args, the array parameter to A itself
t <- evaluate(target)                           -- must fold to a number, else none
for each assignment of variants to D (at most 16):
    for r in 0 .. n-1:
        decode(i) := variant(A[(i - offset + r) mod n])    -- reimplemented decoder
        if evaluate(E, decode) == t: return r              -- first match, as the loop
return none                                     -- consensus or unresolved, below
```

The procedure rests on three conditions: the decoder is identified with its correct variant and offset; the restricted evaluator computes `E` and the target as JavaScript would; and each failed iteration of the loop performs exactly one left rotation. Under these conditions, the loop and Algorithm 1 compute the same sequence of values. The loop rotates the stored array one step after each mismatch, and it also rotates when the body throws. The algorithm evaluates the same expression in the same order for `r = 0, 1, …` and stops at the same first match. Rotations are equivalent modulo `n`, so if no `r < n` matches, the loop never terminates and the algorithm returns none. The conditions can fail. An unrecognised rotation step breaks the third condition (Section 5.4), and an expression the evaluator cannot compute is treated as a mismatch, so the program's rotation may be skipped. A skipped rotation yields none unless a later rotation also matches. A wrong rotation rarely produces a number: it places non-numeric strings at the positions `parseInt` reads, and the sum becomes NaN. The cost is at most 16·n evaluations of `E`, each capped at 20,000 evaluator steps; in obfuscator output, `E` contains six to twelve decoder calls.

A worked example shows the procedure on a three-line program obfuscated by javascript-obfuscator 5.9.0 with rotation enabled (seed 19, plain strings; reformatted, with the declarations reordered for reading):

```
const cp = require("child_process");             // input
const url = "https://example.invalid/setup";
console.log(url, typeof cp.spawn);

function W(){const a=['416VKoFbO','68MVpWjx','spawn','3664340cHkggg','272844EERGGb',
  '2935488qIidkn','28811hhXjYz','2767497Wyegbc','https://example.invalid/setup',
  '3715170urlFCz','5448065HFAxTT','log','child_process'];W=function(){return a;};return W();}
function f(G,d){G=G-0x107;const t=W();let E=t[G];return E;}
(function(G,d){const t=G();while(!![]){try{
  const E=-parseInt(f(0x110))/0x1*(-parseInt(f(0x10b))/0x2)+parseInt(f(0x111))/0x3
    +parseInt(f(0x10d))/0x4+parseInt(f(0x113))/0x5+-parseInt(f(0x10f))/0x6
    +-parseInt(f(0x107))/0x7+parseInt(f(0x10a))/0x8*(-parseInt(f(0x10e))/0x9);
  if(E===d)break;else t['push'](t['shift']());
}catch(L){t['push'](t['shift']());}}}(W,0xaf1a1));
const cp=require(f(0x109)),url=f(0x112);
console[f(0x108)](url,typeof cp[f(0x10c)]);
```

`W` is a self-redefining array source with 13 strings. Eight of them are numeric tokens that the obfuscator added for the checksum. `f` indexes `W` after subtracting `0x107` (263) and has no alphabet or key, so the variant is plain. The immediately invoked function receives `W`, contains `t['push'](t['shift']())`, and its `try` block declares `E` and tests `E === d`. Binding `d` to the call argument gives the target 717,217. For `r = 0 … 9`, at least one `parseInt` call reads a non-numeric string and `E` is NaN. At `r = 10` every `parseInt` call reads a numeric token, and `E` equals 717,217 exactly; no later rotation matches. With `r = 10`, `f(0x109)` decodes to `child_process`, `f(0x112)` to the URL, and `f(0x108)` and `f(0x10c)` to `log` and `spawn`, which restores the original program. An independent reimplementation of the checksum in Python gives the same single solution.

When the checksum cannot establish a rotation, recovery follows three rules:

- *No rotation code:* the array is taken as stored ("unrotated").
- *Rotation code but no solvable checksum:* the scanner falls back to *consensus*. It accepts a rotation only if that rotation decodes at least 80% of up to 64 call sites to printable text, and the runner-up decodes at most half as many. A keyless base64 or plain array decodes to printable text under every rotation, so consensus cannot select one and the array is reported as unresolved.
- *Never assumed unrotated:* an array is never treated as unrotated when recognised rotation code exists (Section 5.4 shows an unrecognised rotator that defeats this in the evaluated engine; Section 5.6 describes the change that closes it). An earlier implementation missed a parameterless `for(;;)` loop and incorrectly decoded 22 commercial-bundle files under rotation 0. This failure illustrates why plausible output alone cannot establish the correct rotation.

Call sites are calls to a decoder or to a helper that reaches one through single-return wrappers and `const` aliases. Calls inside wrappers are excluded as plumbing. Each site is evaluated, and arguments drawn from constant objects resolve. Recovered strings are substituted wherever rules read literal values, including concatenations of decoded fragments.

3.4 Tracing copied values

Clipboard writes (`navigator.clipboard.writeText`, `clipboardData.setData`) are analysed in three steps:

1. The written value is evaluated. When it resolves, the exact text is classified.
2. Otherwise, string fragments are harvested, not evaluated, from the value expression, from locally defined helper functions and objects of helpers (to a bounded depth and node budget), and, when the value is a parameter of a generic copy helper, from the arguments its callers pass.
3. The text is classified by indicator: pipes into a shell, download cradles, hidden or encoded PowerShell, `iex`, `mshta`, `cmd /c`, base64 decoding, detaching (`nohup`, `setsid`, `perl -MPOSIX`), `osascript`, quiet `msiexec` and `rundll32`.

A finding is an error when a strong indicator is present (hidden or encoded window, `mshta`, base64 decoding, detaching). It is also an error when the write occurs inside a `copy`/`cut` handler or carries

verification-lure wording. A plain `curl … | sh` behind a copy button is a warning, because installation pages do exactly that.

3.5 Severity by combination

Each rule has a default severity and explicit escalation conditions. The calibration principle is that a technique legitimate software uses routinely may not be an error on its own:

- *Electron launches.* Launching a process with `ELECTRON_RUN_AS_NODE` in its environment is a warning, and assigning the variable is a note. Either escalates when the launching function, or a local helper it calls, also hides execution through a hidden-window PowerShell invocation or a script launched from a temporary or `.vscode` directory.
- *Obfuscation.* A string array is a warning whose detail records whether it was recovered.
- *Encrypted loaders.* An encrypted loader is an error. It requires embedded base64, AES-GCM authentication and decoded text passed to `Module._compile`, linked through constant bindings.
- *Rules derived from the case.* Investigation-specific markers are notes.

3.6 Isolation

The command-line tool analyses each file in a pool of child processes. Each process runs as follows:

- under the Node.js permission model, with no file writes, child processes, workers or addons, and reads limited to the scanner, its parser dependencies and named rule files;
- with code generation from strings disabled;
- with a 512 MiB heap and an empty environment;
- with a per-file deadline that kills and replaces the process;
- on Linux, where unprivileged user namespaces are available, inside an empty network namespace with `prlimit` caps on CPU time, file size and open files.

The applied controls are recorded in the output. A hostile rule module that attempts a file write, a process launch and a network connection is refused all three in our tests. These controls limit resource use and access if analysis fails. The tests establish the listed restrictions, not containment of arbitrary parser or engine exploits.

4. Implementation

`unk-scanner` is written in JavaScript for Node.js 22 or later, with Acorn 8.17.0 and `@babel/parser` 8.0.8 pinned. It is MIT licensed and published on npm and PyPI; the Python wheel bundles the engine. The engine has about 2,500 lines in fifteen modules: the parser front end, evaluator, string-array recovery, loader recognition, isolation pool, command-line interface, SARIF writer and the rule modules. Rules are modules with an identifier, default severity, ATT&CK mapping and a `create(context)` function returning visitors, in the style of ESLint. All visitors are dispatched in one pass over the indexed tree. The nine built-in rules are grouped in four modules: clipboard, Electron launch, obfuscation and loaders, and campaign markers. A custom rule receives the same context, including resolved values, harvested fragments and recovered strings, and loads through `--rule` or the library interface.

The scanner's own tree walks are iterative with explicit stacks and node budgets, so deeply nested input exhausts a budget rather than the call stack. Only the parsers recurse, and a parser stack overflow is reported as a failed file. Positions are computed on demand from a line index rather than from Acorn location objects, which reduced peak memory on a 7.9 MB bundle from 462 MB to 399 MB. A report records which parser accepted the file, each recovered array (variant, offset, rotation, method, decoded and total call sites), the isolation controls applied, and a fixed list of limitations, so a reader can see what was attempted as well as what was found. Output is JSON (schema 2) or SARIF 2.1.0 with `security-severity` properties, so findings appear as

code-scanning alerts. The current test suite has 67 cases, including the bypass fixtures of Section 5.4 as regression tests, and runs on Linux, macOS and Windows with Node.js 22 and 24. The accompanying repository records the evaluated source fingerprints, package metadata and test suite [2].

5. Evaluation

The evaluation combines scans of popular software, recently changed npm packages and known-malicious samples. Together they measure alert volume and detection coverage; retained campaign artifacts and single-change variants test string recovery and resistance to evasion. Parser coverage and throughput establish the practical cost of analysis. A controlled experiment with known originals (Section 5.7) measures the correctness of string recovery and compares it with three deobfuscators.

5.1 Corpora

Tier	Corpus	Purpose
Calibration	2,500 npm (`npm-high-impact` 1.13.0 ranking) and 500 Open VSX extensions by downloads; 143,153 unique files	Used during development to set severities; not held out
1	Top 5,000 npm packages (`npm-high-impact` ranking; 4,992 acquired) and top 1,000 VS Code Marketplace extensions by installs (967 acquired), with the calibration Open VSX set; 332,054 unique files, 188,901 not in the calibration corpus	Alert volume beyond the initial calibration set; one rule adjustment informed by Tier 1
2	20,000 most recently changed npm packages from the registry changes feed (sequences 140,571,698–140,654,245, 29 October 2026); 19,976 acquired, 2,403,437 unique files; 84% scoped	Discovery in the unvetted stream
3	Datadog malicious-packages dataset at commit `438afad` [3]: all 122 IDE-extension samples and a seeded random sample (seed 20261029) of 2,000 of 26,715 npm samples, 41,359 unique files; the 14 retained UNK_DeadDrop artifacts	Sensitivity and bypass analysis

Archives are read in memory under size caps; nothing is installed and no lifecycle script runs. Malicious samples remain encrypted at rest, and their members are written only as inert files for parsing.

5.2 Findings in popular software

On the calibration corpus no error-level finding occurred in 143,153 files. All 107 warnings were reviewed:

- 90 were commercial obfuscation, in five packages, all recovered or reported as unresolved;
- 16 were legitimate launches of the editor binary as Node, among them four bundled copies of TypeScript's typings installer;
- one was an installer's copy button.

Because severities were set on this corpus, it shows consistency rather than generalisation. Rescanned with the frozen engine on the evaluation server, the corpus again produced no error and 108 warnings; the additional warning is a TypeScript browser-installer helper in an AI coding extension that the earlier engine could not parse, and it is a legitimate launch. Tier 1 extends the measurement beyond the initial calibration set. Its 332,054 unique files include 188,901 that do not occur in the calibration corpus, drawn from 2,783 packages and extensions. The engine was frozen before the final run (source hash recorded with the results). On those 188,901 additional files, the final engine raised no errors and 12 warnings; all warnings were reviewed:

- 11 were legitimate launches of the editor binary as Node: helper and port-forwarder launch templates in Microsoft Live Share and GitHub Codespaces, three bundled copies of TypeScript's typings installer, and a theme patcher's safety check and documented uninstall hook;
- one was a commercial theme's licence code, an RC4 string array fully decoded under the checksum-proven rotation, whose hidden strings are licence-shop and update URLs and favicon-based connectivity probes.

An initial Tier 1 run, before the engine was frozen, raised four errors on the Live Share and Codespaces files. The bundles also contain a copy of a privilege-elevation library that runs PowerShell with a hidden window, and the escalation then applied to the whole file. The rule now escalates only when the hidden execution occurs in the launching function or a local helper it calls; this was the only change made after examining Tier 1 results, and it preserved every retained artifact's previous severity. Consequently, the additional files are disjoint from the initial calibration set, but the final result is not an untouched held-out estimate.

5.3 Recently changed npm packages

Tier 2 is the unvetted stream: every package whose registry document changed in a window of about 83,000 change sequences ending on 29 October 2026, most of them published that day. The scanner raised 2 error-level and 6,215 warning-level findings in 2,403,437 files from 19,976 packages:

Rule	Severity	Findings	Packages
`string-array-obfuscation`	warning	6,142	153
`electron-run-as-node`	warning	68	44
`clickfix-clipboard-command`	warning	3	1
`child-process-hidden-script`	warning	2	2
`electron-run-as-node`	error	1	1
`clickfix-clipboard-command`	error	1	1

Full review classified both error-level findings as false positives. An AI desktop application builds a hidden, encoded PowerShell command that sets `ELECTRON_RUN_AS_NODE` for its own helper, which only contacts `127.0.0.1` with a local password: the technique combination the rule escalates on, used for a legitimate purpose. A remote-access tool copies a setup guide whose only `curl` call prints an HTTP status code; the word "verify" in the guide supplied the lure indicator. The clipboard, hidden-script and Electron warnings were all reviewed: the code was read for the clipboard and hidden-script warnings and for five Electron packages whose names did not suggest developer tooling, and the remaining 39 Electron packages were triaged from the recorded launch target and environment. The reviewed contexts were consistent with legitimate helper relays, licence checks, launcher shims, desktop-app smoke tests and bundled TypeScript. For the 153 obfuscated packages, the strings recovered from their arrays were screened for process, shell, exfiltration-service, wallet and credential-store indicators: seven packages matched, and each is explained by its function (a Telegram and Discord bot, an OAuth-based workspace connector, agent CLIs); nine packages kept unresolved arrays, eight of them modules of one commercial product. No package was classified as malicious or reported to the registry. This triage outcome does not establish that every package is benign, particularly where arrays remain unresolved.

Weekly downloads were retrieved in bulk for unscoped names and individually for every flagged scoped package and a seeded sample of 500 other scoped packages. Flagged packages are not concentrated in the long tail: of the 168 flagged scoped packages, 7 had fewer than 100 weekly downloads and 65 had 10,000 or more, against 80 and 87 in the 500-package sample. These counts

show that alerts also arise in widely used tooling. At error level, 2 of 19,976 packages (0.01%) required review. This is the observed package-level review rate, not a population false-positive estimate.

5.4 Detection coverage, recovery and bypasses

On the retained UNK_DeadDrop artifacts, all three ClickFix lure generations, three encrypted loaders, three decrypted collectors and the Windows Node/Python pipeline reach error level. The two obfuscated extension files are recovered and reported as warnings. The Windows Node agent produces only notes, a miss: its collection code carries no launch, clipboard or loader structure the rules key on. The login interface, which is context-only by design, produces none. These artifacts informed the rules, so they establish coverage, not generalisation.

The checksum method gives rotation 391 for the extension and 261 for its bootstrap, the values established manually during the investigation by a printable-output comparison, and decodes 2,139 of 2,140 and 15 of 17 call sites.

Across the calibration corpus, 486,208 of 486,601 decoder call sites in 90 arrays were decoded. Of those arrays, 71 were solved by checksum (69 plain, 1 RC4, 1 shared base64 and RC4) and 19 were left unresolved. The unresolved arrays were reached through reassignable `var` aliases, which the evaluator does not follow. The artifact and calibration figures were reproduced exactly by the frozen engine on the evaluation server.

Tier 3 measures sensitivity on malware outside the campaign, and the result bounds the scope of the approach. A sample counts as detected when any of its files raises a warning or error:

Sample set	Samples	Error	Warning	Not detected	No JavaScript
IDE extensions	122	0	4	113	5
npm, malicious intent	1,884	0	44	1,550	290
npm, compromised library	116	0	14	102	0

Detections came from string-array obfuscation (55 samples) and hidden-script launches (7); no sample reached error level, and none used the ClickFix, Electron or encrypted-loader structures the higher-severity rules describe. Parser failure explains only 27 misses. To characterise the remaining 1,738, their files were screened for common static traits (these traits also occur in benign code, so the counts describe the misses rather than establish causes). Among missed npm samples, 900 read environment variables or home-directory files, 873 use `child_process`, 871 make network requests and 397 reference a webhook or messaging exfiltration service. Among missed IDE extensions, 81 use `child_process` and 21 contain variation-selector code points of the kind used to hide payloads in invisible Unicode. The missed samples expose a major coverage gap: credential-access and install-time behaviours can appear without the clipboard, Electron or encrypted-loader structures modelled here. The retained campaign artifacts demonstrate recognition of the structures used to design the rules, but cannot establish generalisation even within recruitment campaigns. Their 14 expected outputs include both the missed Windows agent and the context-only login interface.

To measure how easily the rules are evaded by an adversary who knows them, we wrote inert variants of the demonstration fixtures and of the loader test fixture, each changing one thing, and scanned them with the CLI on the evaluation server:

Technique (baseline)	Variant	Result
ClickFix copy (error)	builder through a `var` alias; builder named via `eval`; command in base64 decoded with `atob`; command as character codes spread into `String.fromCharCode`; command in a mutable variable set elsewhere; copy through a hidden textarea and `execCommand`; command fetched at copy time	all missed
Electron launch (error)	variable name split into constant fragments; name base64-decoded at run time	missed
Electron launch (error)	editor binary spawned directly with `windowsHide`; hidden window via encoded PowerShell	warning
Encrypted loader (error)	same steps inside a function; `vm.runInThisContext` instead of `_compile`; AES-CBC without tag; `let` bindings	all missed
String array (warning, recovered)	wrapper reached through a `var` alias or an object property	warning, but the aliased call sites (the URL and module name) not decoded
String array (warning, recovered)	checksum target not foldable	recovered by printable consensus
String array (warning, recovered)	rotation moves elements with `splice` instead of `push(shift())`	incorrect: rotator not recognised, decoded under rotation 0

Eighteen of nineteen variants evade the rule they target, lower its severity or defeat string recovery; only the unfoldable checksum target is handled, by the consensus fallback. The clipboard, Electron and loader rules recognise observed structures but remain sensitive to small rewrites. The last row is a defect: the design rule of Section 3.3 holds only for rotation code the scanner recognises, and an unrecognised rotator falls back to the unrotated decoding the rule exists to prevent. The results in this section use the unchanged engine; Section 5.6 evaluates a hardened engine against the same variants and corpora. Partial recovery is reported as recovered, so an analyst must compare decoded with total call sites.

5.5 Throughput and parser coverage

All measurements in this paper were taken on one machine, an eight-vCPU virtual server with 15 GiB of memory, with the frozen engine and four analysis workers unless stated. On the calibration corpus the isolated CLI path processed 272 files/s (7.4 MB/s) and in-process worker threads 264 files/s; the isolated run was slightly faster in this comparison. Repeated trials would be needed to estimate the overhead and its variance. Median analysis time per parsed file was 1.6 ms, p99 207 ms and the maximum 12.5 s, for an 8 MB bundle. Malicious Tier 3 samples, scanned only on the isolated path, ran at 325 files/s, and Tier 2 at 848 files/s with six workers, reflecting its many small files. Before TypeScript support, Acorn rejected 31,964 calibration files; with the Babel fallback 352 remain. Tier 1 parsed 330,473 of 332,054 files (99.5%): 251,593 by Acorn, 77,521 as TypeScript, 668 as TSX, 572 as Flow with JSX and 119 as JSX. Of the 1,576 failures, 1,305 are TypeScript files, including legacy `module X {}` namespace syntax. None of the failed inputs parsed under the independent Babel configuration tested; this does not establish that all are invalid in their original toolchains. Five files are not valid UTF-8. Tier 2 parsed 2,398,405 of 2,403,437 files (99.8%).

5.6 Hardening after the evaluation

After the frozen-engine measurements, we changed the engine to absorb the bypass variants and rescanned every corpus and the retained artifacts on the same server; the hardened engine's source hash is recorded with its results, separately from the frozen engine's. The changes generalise the structures rather than adding the variants as signatures:

- the evaluator treats `let` and `var` bindings as constants when no assignment in the file resolves to them, and evaluates `atob`, `Buffer.from(...).toString(...)`, spread arguments of known arrays and `eval` of a constant identifier name (other `eval` text is still not interpreted);
- clipboard tracing harvests every value stored into a mutable variable, recognises copy handlers registered as object methods, and treats `document.execCommand('copy')` as a write of the field filled before it;
- the Electron rule resolves script text whose variable name is assembled or decoded, counts encoded PowerShell as hidden execution, and escalates a detached, windowless launch of a binary other than the editor's own runtime;
- loader recognition links an embedded encoded payload, a recognised cipher in any mode and a code-compiling sink (`Module._compile`, `vm`, `eval`, `Function`) through bindings in any scope;
- rotation recognition accepts equivalent steps (`push(splice(0, 1)[0])`, `unshift(pop())`), treats any other reordering in the rotation function as rotation code, accepts the unrotated order only when at least 80% of sampled call sites decode to printable text, and follows call sites through aliases and object members.

With these changes, 18 of the 19 variants reach the baseline's severity and full string recovery; the splice-rotated array now decodes to the baseline strings under the checksum-proven rotation. The exception is the command fetched at copy time, which is not in the file. The retained artifacts again match their 14 expected outputs, and the test suite now includes every variant.

Corpus	Files	New errors	Files with new warnings	Change in string recovery
Calibration	143,153	0	2 (1 package)	90 arrays (19 unresolved) to 96, all solved by checksum; 660,545 of 660,678 call sites decoded
Tier 1	332,054	0	2 (the same package; none in the 188,901 additional files)	19 unresolved arrays to none
Tier 2	2,403,437	0	3 (2 packages)	1,387 unresolved arrays to 16; packages with no resolved array from 9 to 0
Tier 3	41,359	0	3 (2 samples)	detected samples from 62 to 64, none at error level

The new warnings are a commercial extension's genuine obfuscator.io arrays and an AI scripting CLI's string arrays, both declared with `var` and previously skipped, and an installer's `curl … | bash` copy button. The newly decoded Tier 2 strings were screened as in Section 5.3. Two packages newly revealed a `child_process` import: a commercial SAP CAP plugin whose licence code downloads an encrypted per-customer configuration and parses it as JSON, and a commercial developer CLI whose launches list files with `git` and locate the global npm directory. Static reading found no code-execution sink for downloaded data, and neither package was classified as malicious. The Univer Pro modules and the agent library whose arrays had stayed unresolved decode to product-licence and service endpoints and public DNS-over-HTTPS resolvers.

The hardening also reproduced the calibration problem in Section 5.2. A first version escalated any detached Electron launch without a window or stdio, which raised an error on an AI coding extension that relaunches its own runtime (`process.execPath`) as a detached helper server. The rule now escalates only a named binary other than the extension's own runtime. A malicious extension

using the same relaunch therefore remains a warning: the syntax does not distinguish them. On the calibration corpus with four in-process workers, the hardened engine processed 281 files/s (median 1.2 ms, p99 210 ms per file), against 264 files/s for the frozen engine in the same configuration, so the added evaluation has no measurable throughput cost.

These figures show that the evaluated bypasses were shallow and can be closed without measurable alert-volume cost. They do not show robustness. The variants were written by the authors, the hardening was designed against them, and an adversary who knows the new rules can choose the next rewrite: a reassigned alias, an unrecognised sink, or text moved out of the file.

5.7 Controlled string-array recovery

The corpus figures count decoded call sites but cannot say whether a decoded string is the one the program would use, because real obfuscated code has no ground truth. A controlled test supplies one. Fifty benign files were drawn from the calibration corpus by seeded sample, one per package, each a script of 3–30 KB with at least 15 distinct strings and no findings. Each file was obfuscated with javascript-obfuscator under 72 string-array configurations: three encodings (none, base64, RC4), rotation on or off, no wrappers or two variable or function wrappers, calls transform on or off, and index shift on or off. Each sample has its own seed. Two obfuscator releases were used: 5.9.0, current at the time of the test, and 4.1.1, current when the newest baseline below was last updated. Only string-array transforms were enabled, so each hidden string (a string literal, template text or property name of the original that no longer appears outside the obfuscator's array) has a known original.

Both scanner engines processed all 7,200 samples. Three deobfuscators were run for comparison on a stratified subset of 1,440 samples (every configuration and both releases, first ten inputs), one process per sample with a 60 s limit: webcrack 2.16.0 [9], which runs the decoder in a sandbox, and the static tools synchrony 2.4.6 [8] and obfuscator-io-deobfuscator 1.0.6 [10]. Scoring is identical for every tool. A hidden string is recovered when it appears in the tool's output, or, for the scanner, among its decoded call-site values. An output string counts as unexplained when neither the original nor the obfuscated sample contains it, either whole or as a concatenation of their strings; deobfuscators legitimately fold `'a' + 'b'`. Set scoring cannot detect a wrong rotation of a plain or base64 array, because every call site would then decode to another genuine string of the same program. The scanner's results are therefore also checked site by site. The obfuscator replaces each string occurrence in place, so the decoded call sites, read in source order, must align with the original's sequence of strings, and sites outside the longest common subsequence count as misplaced. As a validation, decoding 25 unencoded samples correctly gives no misplaced sites, while shifting every decoded value by one array position, as a wrong rotation would, misplaces 4,240 of 4,903 sites.

Tool (samples)	Recall, 5.9.0	Recall, 4.1.1	Samples with unexplained strings
`unk-scanner`, frozen (7,200)	68.3%	0%	0
`unk-scanner`, hardened (7,200)	87.2%	0%	0
`unk-scanner`, hardened (subset, 1,440)	90.3%	0%	0
webcrack (1,440)	98.1%	98.1%	144, one input
synchrony (1,440)	0%	98.2%	144, one input
obfuscator-io-deobfuscator (1,440; 144 timed out)	0%	98.0%	144, one input

The scanner never produced an unexplained string in either engine. The site-level check also passed throughout. All 638,574 call sites decoded by the hardened engine, and all 492,474 decoded by the frozen engine, align with the original program in source order. Every string it reported was

the program's own and in its place. Its misses were reported as unresolved call sites or no recovered array. On 5.9.0 output, the hardened engine solved the rotation by checksum in all 1,800 rotated samples, against 1,344 for the frozen engine. Without rotation code, it accepted the stored order in 1,602 of 1,800 samples and left the rest unresolved. Recall was at least 99.7% in every configuration except one combination. When function wrappers are combined with the calls transform, recall falls to 24%: each wrapper then declares a local offset object before returning, which falls outside the single-return helper model of Section 3.2, so call sites through those wrappers stay unresolved. The frozen engine's lower recall comes from wrapper aliases declared with `var` (64%) and from the rotations it could not solve. Neither engine recovers anything from 4.1.1 output. That release's decoder replaces itself with an inner function, which rebases the index rather than the outer decoder's parameter (Section 3.3), so no decoder is identified, and the scanner reports no string array rather than guessing.

The baselines show complementary failures. On 5.9.0 output, synchrony locates the array but stops with "no decoder", and obfuscator-io-deobfuscator returns the code with the array in place; both recover over 98% of 4.1.1 output. webcrack recovers 98% of both releases, because it executes the decoder instead of matching its shape. Its remaining misses are rewrites, not failed decodings: strings folded into longer literals, a constant `JSON.parse` turned into an object literal, and an empty string. The unexplained strings of each baseline come from a single input. webcrack's are the keys of that rewritten JSON object. synchrony turns an obfuscator variable name into a property name. obfuscator-io-deobfuscator's constant propagation replaced a variable with `undefined` and folded it into an error message, which changes the program's output. The synchrony and obfuscator-io-deobfuscator misses were not inspected string by string. The median time per sample, including parsing and scoring, was 30 ms for the scanner and 290–1,045 ms for the baselines, which also regenerate code.

The test supports a narrow version of the title's claim. When the scanner identified the decoder and the rotation loop, every decoded call site was correct in all 7,200 samples. When it does not, it says so. Coverage is a separate matter: a current obfuscator release with one option combination, and an older release's decoder shape, defeat recognition entirely. The sandboxed approach covers both, at the cost of executing input-derived code and rewriting the program. The test uses one obfuscator family on benign inputs. Malware authors also apply control-flow flattening and dead-code injection, which were disabled here, and custom obfuscators.

6. Case study: UNK_DeadDrop

The retained UNK_DeadDrop artifacts illustrate the three analysis tasks [1]. ClickFix lures assembled detached `curl` and `perl -MPOSIX` commands at copy time. In one generation, the clipboard write appeared inside a generic `copyText(text)` helper; tracing its callers exposed the command builder. The extension instead concealed module names, including `child_process`, and key material in an RC4 string array. Recovering the array made those values available for inspection. The Windows branch generated hidden PowerShell that launched collection code through the signed editor binary with `ELECTRON_RUN_AS_NODE`.

The retained artifacts and their results are listed below. The frozen and hardened engines produce identical outputs on all 14.

Retained artifact	Expected	Result	Rule
ClickFix lure bundles and script (3 generations)	error	error	`clickfix-clipboard-command`
Encrypted stage-2 loaders (3)	error	error	`encrypted-loader`
Decrypted stage-3 collectors (3)	error	error	`electron-run-as-node`
Windows Node/Python pipeline	error	error	`electron-run-as-node`

Retained artifact	Expected	Result	Rule
VSIX extension entry point and bootstrap	warning	warning	`string-array-obfuscation`, recovered
Windows Node agent	note	note	none: a known miss
Panel login interface	none	none	context only

These structures motivated clipboard tracing, decoder recovery and combined launch indicators. They also explain the evaluation's limits: the rules recognise particular implementations recovered from the case, while the Windows agent's collection code remains below warning level and small rewrites defeat several detectors. Analysts can use recovered strings and call sites to guide examination, but an absence of findings does not clear an artifact.

7. Discussion and limitations

Static analysis exposes code structure and configured actions, not execution or impact. This scanner does not prove reachability, reconstruct the browser's displayed text or recover code fetched at run time. Its evaluator handles only the documented subset; reassignable aliases, dynamic code and unsupported encodings can prevent recovery. Even within obfuscator.io-style code, an unrecognised rotation loop can produce incorrect strings rather than an explicit failure. Partial recovery must therefore be assessed against the number of unresolved call sites.

The corpora answer different questions. Popular packages measure noise in widely distributed software, but are a convenience sample with correlated files. Tier 1 also informed one rule adjustment before the final scan, so its final result is not an independent validation of that rule. The new-package stream measures review burden over one registry window, while the public malware dataset reveals poor sensitivity outside the observed campaign structures. None establishes a deployment-wide false-positive rate.

Manual review was complete for error-level findings. Warning review used code inspection, recorded launch context and recovered-string screening as described in Section 5.3; it was not a comprehensive audit of every flagged package. Unresolved arrays and metadata-based triage leave uncertainty about those packages. The bypass study further shows that low alert volume can coexist with easy evasion. Closing the evaluated variants (Section 5.6) did not add error-level alerts, but it was tested only against variants known in advance.

Samples were parsed without execution or installation, and acquisition did not run package lifecycle scripts. No Tier 2 package was classified as malicious, so no registry report resulted from that review. Public artifacts provide manifests, hashes and review records without redistributing the source corpus.

8. Related work

GuardDog [5] combines Semgrep and YARA rules over package source with metadata heuristics across ecosystems, including npm and VS Code extensions. Amalfi [6] classifies npm packages using code, metadata and change features. Ohm et al. [7] catalogue software supply-chain attacks and their injection and execution techniques. These approaches address behaviours prominent in the public malware corpus, where the rules evaluated here have low sensitivity. Combining delivery-specific inspection with install-script and credential-access analysis is therefore more plausible than treating either as complete coverage.

JavaScript deobfuscators differ in how they recover string arrays. webcrack [9] regenerates the sample's array, decoder and rotation code and runs it in an `isolated-vm` sandbox. synchrony [8] and obfuscator-io-deobfuscator [10] instead match decoder templates and reimplement them; synchrony also evaluates the `parseInt` checksum chain to determine rotation. Our recovery follows this static approach, sharing a restricted evaluator with the detection rules and presenting recovered values as

triage evidence rather than rewriting the file. Section 5.7 compares all four on the same obfuscated samples. Each static tool covers a different set of decoder shapes and fails on the others. webcrack's sandboxed execution covers both obfuscator releases tested. The scanner produced no unexplained strings, but its recognition limits are material. Section 5.4 demonstrates incorrect recovery when a rotator falls outside the supported syntax, and Section 5.7 shows decoder shapes it does not identify at all.

Proofpoint [11] and Microsoft Threat Intelligence [13] describe ClickFix lures that induce users to paste commands into a terminal or run dialog. Unit 42 [12] reported developer targeting through fake interviews and npm-based tasks under Contagious Interview, now tracked as ATT&CK G1052 [4]. The present work studies static inspection of related delivery mechanisms; it does not resolve the attribution of UNK_DeadDrop to that group.

9. Conclusion

`unk-scanner` recovers commands and configuration from recognised JavaScript structures without executing the input. Its bounded evaluator supports clipboard tracing and checksum-based string-array recovery, while combined indicators reduce error-level alerts on ordinary developer tooling. The evaluated CLI processed 272 calibration files per second with process isolation.

Low alert volume comes with substantial limits. The scanner flagged only 62 of 2,122 public malware samples, and 18 of 19 single-change variants evaded a rule, lowered severity or defeated recovery. One variant produced incorrect decoded strings. In a controlled test with known originals, the scanner's decoded strings were always the program's own, but it recognised the decoders of only one of two obfuscator releases. A hardened engine closes all but one of those variants without new error-level findings on 2.8 million files, but that measures the absorbed variants, not an adaptive adversary. These results support using the tool to expose evidence for further inspection, alongside broader package and credential-access analysis. They do not support using a clean scan as a safety decision. Published manifests, review records and bypass fixtures make those limits reproducible and provide a basis for testing future changes.

Artifact availability

Source, rules, inert fixtures, benchmark harness, acquisition manifests, review ledgers and the controlled recovery experiment (`benchmark/rotation`, with pinned tool versions, sample manifests and per-sample results) are in the companion repository [2]. Corpus source is not redistributed; selection manifests and hashes allow it to be reacquired.

References

[1] Ilja Krasnjanski. UNK_DeadDrop: An Artifact-Based Study of Recruitment Malware, Credential Collection, and Operator Infrastructure. Zenodo preprint. https://doi.org/10.5281/zenodo.23652297

[2] KrasForge. `unk-deaddrop-companion` and `unk-scanner`. https://github.com/KrasForge/unk-deaddrop-companion

[3] Datadog Security Labs. Open-Source Dataset of Malicious Software Packages. https://github.com/DataDog/malicious-software-packages-dataset (Apache-2.0).

[4] MITRE ATT&CK. Contagious Interview, G1052. The MITRE Corporation, 2025. https://attack.mitre.org/groups/G1052/

[5] DataDog. GuardDog. GitHub, Apache-2.0. https://github.com/DataDog/guarddog

[6] Adriana Sejfia and Max Schäfer. Practical Automated Detection of Malicious npm Packages. In Proceedings of the 44th International Conference on Software Engineering (ICSE '22), ACM, 2022, pp. 1681–1692. https://doi.org/10.1145/3510003.3510104

[7] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. In Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2020), LNCS 12223, Springer, 2020, pp. 23–43. https://doi.org/10.1007/978-3-030-52683-2_2

[8] relative. synchrony: javascript-obfuscator cleaner and deobfuscator. GitHub, GPL-3.0. https://github.com/relative/synchrony

[9] j4k0xb. webcrack. GitHub, MIT. https://github.com/j4k0xb/webcrack

[10] ben-sb. obfuscator-io-deobfuscator. GitHub, Apache-2.0. https://github.com/ben-sb/obfuscator-io-deobfuscator

[11] Tommy Madjar, Selena Larson, and the Proofpoint Threat Research Team. Security Brief: ClickFix Social Engineering Technique Floods Threat Landscape. Proofpoint, 18 November 2024. https://www.proofpoint.com/us/blog/threat-insight/security-brief-clickfix-social-engineering-technique-floods-threat-landscape

[12] Unit 42. Hacking Employers and Seeking Employment: Two Job-Related Campaigns Bear Hallmarks of North Korean Threat Actors. Palo Alto Networks, 21 November 2023. https://unit42.paloaltonetworks.com/two-campaigns-by-north-korea-bad-actors-target-job-hunters/

[13] Microsoft Threat Intelligence and Microsoft Defender Experts. Think before you Click(Fix): Analyzing the ClickFix social engineering technique. Microsoft Security Blog, 21 August 2025. https://www.microsoft.com/en-us/security/blog/2025/08/21/think-before-you-clickfix-analyzing-the-clickfix-social-engineering-technique/